

SaaS2 — Clothes

ملخص شامل لما تم تنفيذه حتى الآن

Multi-Tenant Clothing Inventory & Sales SaaS

الحالة الحالية	الأساس المعماري، المصادقة، الموافقات، Categories، والSale/Return/Void/Exchange lifecycle جاهزة
التطوير الحالي	Local frontend/backend + Supabase PostgreSQL/Auth
الصور	Cloudflare R2 مُجهّز، وSharp/R2 integration بعده لم يبدأ
آخر نقطة قيد العمل	تصحيح وإكمال DailyReport accounting redesign قبل الdeploy

آخر تحديث: 15 سبتمبر 2026

1. الحالة العامة للمشروع

SaaS صار عنده أساس قوي ومتكامل للمصادقة، تعدد المتاجر، الموافقات، إدارة التصنيفات، وبنية مالية/مخزنية تحفظ التاريخ وتدعم الإلغاء والترجيع والتبديل. المنتجات والـPOS بعدهم الخطوات التالية بعد تثبيت DailyReport.

الحالة	المكوّن
مكتمل ✓	Authentication + Supabase Auth
مكتمل ✓	Multi-tenant roles & tenant isolation
مكتمل ✓	OWNER signup/onboarding
مكتمل ✓	SUPER_ADMIN bootstrap/login
مكتمل ✓	Account approval lifecycle
مكتمل ✓	Admin approval dashboard
مكتمل ✓	Authenticated business shell
مكتمل ✓	Categories backend + frontend
مكتمل ✓	Product imageKey schema
مكتمل ✓	sellingPrice rename
مكتمل ✓	averageCost removal
مكتمل ✓	Sale/Return/Void/Exchange DB lifecycle
مُجهّز ✓	Cloudflare R2 bucket/token setup
قيد الإكمال ⏳	DailyReport redesign
لم يبدأ ⏳	Sharp + R2 integration
لم يبدأ ⏳	Products + Variants API/UI
لم يبدأ ⏳	Inventory/Restock runtime
لم يبدأ ⏳	POS/Sales runtime
لم يبدأ ⏳	Returns/Void/Exchange APIs/UI

المكوّن	الحالة
Reports runtime/UI	لم يبدأ ⌚

2. Architecture الأساسية

```

Frontend: React + Vite
Backend: Node.js + Express + TypeScript
ORM: Prisma 7
Database: Supabase PostgreSQL
Auth: Supabase Auth
Images: Cloudflare R2 (planned runtime integration)
Image Processing: Sharp (planned)
Future deployment: Frontend -> Netlify | Backend -> Railway

```

حاليًا الـ frontend والـ backend شغالين Local، بينما Supabase مستخدم للـ PostgreSQL والـ Auth. كل business data يتم تخزينها في PostgreSQL، وليس مباشرة من الـ frontend إلى جداول الأعمال.

3. Multi-Tenant + Roles

```

Account = Store
SUPER_ADMIN -> accountId = null
OWNER        -> accountId = store
WAREHOUSE    -> accountId = store

```

- الـ accountId والـ role لا يُوثق بهما من الـ frontend؛ مصدر الصلاحية هو الـ backend auth context فقط.
- SUPER_ADMIN خارج tenant business APIs
- OWNER وWAREHOUSE يدخلون business shell فقط عندما Account.status = ACTIVE

4. Authentication & OWNER lifecycle

```

/signup
-> Supabase Auth
-> Email confirmation
-> /auth/signup-callback
-> /owner/onboarding
-> POST /api/auth/bootstrap-owner
-> Account = PENDING
-> /pending-approval
-> SUPER_ADMIN approve
-> Account = ACTIVE
-> /app

```

- requireAuth يتحقق من Supabase access token ويأخذ role/accountId من PostgreSQL.
- requireTenant يمنع accounts PENDING / REJECTED / SUSPENDED من business APIs.
- تم تنفيذ end-to-end حقيقي: /app → ACTIVE → SUPER_ADMIN approval → PENDING → OWNER signup.

SUPER_ADMIN .5

- تم إنشاء bootstrap CLI آمن لأول SUPER_ADMIN بدون public HTTP route.
- أول SUPER_ADMIN الحقيقي تم إنشاؤه وتحديد password له، وتحققنا أن role = SUPER_ADMIN و accountId = null و isActive = true.
- Admin dashboard يحمل pending accounts ويتيح Approve و Reject مع reason.

Password / Invite / Recovery .6

```
/auth/callback
/set-password
/login
Forgot password
```

- كلمات المرور تذهب مباشرة إلى Supabase Auth؛ لا تمر عبر Express ولا تُخزن في PostgreSQL.
- تم دعم sign-out، set-password، invite/recovery callback، و login routing.

Business App Shell .7

```
/app/dashboard
/app/products
/app/categories
/app/inventory
/app/sales
/app/expenses
/app/reports
```

- OWNER يرى كل الأقسام.
- WAREHOUSE يرى Dashboard, Products, Categories, Inventory, Sales/POS، ولا يرى Expenses أو Reports.
- في Sidebar/Header/Mobile drawer/Logout لحماية routes عبر /api/auth/me.

8. Backend + Frontend Categories - مكتملة

```
GET    /api/categories
POST   /api/categories
PATCH /api/categories/:categoryId
DELETE /api/categories/:categoryId
```

- Tenant isolation كامل: update/delete باستخدام (id, accountId).
- Duplicate داخل نفس المتجر → CATEGORY_ALREADY_EXISTS.
- Category مستخدمة بـ Product أو CATEGORY_IN_USE → history، بدون cascade delete.
- واجهة app/categories/ تدعم List / Add / Rename / Delete / Confirmation / Loading / Empty / Errors

9. Product Architecture

```
Category
  -> Product
    -> ProductVariant
```

- مثال: Category = Pantalon، Product = Cargo Pants، والـ Variants تكون Black/M، Black/L، Beige/M، Beige/L.
- الـ Product مربوط دائماً بـ Category، والـ stock والسعر والـ SKU يكونوا على مستوى الـ Variant.

10. Product Images

- تم تجهيز Cloudflare R2 bucket خاص بالمنتجات، والـ token محدود على هذا الـ bucket.
- تم تغيير Product.imageUrl إلى Product.imageKey عبر migration data-preserving وتم نشرها على Supabase.

```
Planned image flow:
Frontend upload
-> Express backend
-> validate content/size
-> Sharp resize/compress/WebP
-> Cloudflare R2
-> PostgreSQL stores imageKey only
Canonical key: tenants/{accountId}/products/{productId}/main.webp
```

- R2 + Sharp runtime integration بعده لم يبدأ.

11. ProductVariant contract

```
ProductVariant
- sku
- barcode
- color
- size
- sellingPrice
- lastPurchaseCost
- currentStock
- isActive
```

- suggestedPrice تم تغييرها إلى sellingPrice (السعر الأساسي/catalog price).
- SaleItem.unitSoldPrice يبقى السعر الفعلي الذي انباع فيه المنتج، ويمكن أن يختلف عن sellingPrice.
- averageCost تم حذفه نهائيًا من schema والdatabase.
- lastPurchaseCost هو current cost basis، وSaleItem.unitCostAtSale هو immutable historical cost snapshot.

12. Sale / Return / Void / Exchange Lifecycle

تم تنفيذ ونشر migration كبيرة تحمي التاريخ المالي والمخزني على مستوى PostgreSQL، مع tenant-qualified foreign keys وtriggers وCHECK constraints.

12.1 Void, SaleStatus

```
SaleStatus:
- COMPLETED
- VOIDED
```

- Sale الأصلية لا تُحذف ولا تُعاد كتابة totals تبعها.
- transition الوحيد المسموح بعد الإنشاء هو VOIDED → COMPLETED.
- Void metadata: voidedAt, voidedById, voidedByName, voidedByCode, voidReason
- Void reason مطلوب، وبعد VOIDED تصبح Sale immutable.

12.2 Returns

```
SaleReturn
SaleReturnItem
```

- يدعم Partial Return وFull Return بدون تعديل SaleItem.quantity الأصلية.

- DB: refundAmount = quantity × original SaleItem.unitSoldPrice Refund formula محمية بالـ
- Over-return ممنوع بـ Trigger يستخدم FOR UPDATE ويعيد حساب cumulative returned quantity

Exchange 12.3

Exchange مصمم كـ Return + New Sale مع immutable link، وليس كنوع InventoryMovement مستقل.

Example:
Returned item = \$20
New item = \$25
Difference = \$5 customer pays

Return vs Void mutual exclusion 12.4

- إذا في أي Return على Sale، ممنوع تتحول إلى VOIDED.
- إذا الـ VOIDED Sale، ممنوع إنشاء Return عليها.
- الحماية موجودة بالـ database وتستخدم نفس Sale row lock لتفادي race conditions.

Inventory Movement Lifecycle 13

RESTOCK	+ stock
SALE	- stock
RETURN	+ stock
SALE_VOID	+ stock
DAMAGE	- stock
ADJUSTMENT	+/- stock

- InventoryMovement صار INSERT append-only: مسموح، UPDATE/DELETE ممنوعين.
- Partial unique indexes تمنع duplicate SALE / SALE_VOID / RETURN movements
- RETURN و SALE_VOID unitCost لازم يساوي original SaleItem.unitCostAtSale بالضبط.
- Cross-tenant movement/return references ممنوعة بـ compound foreign keys

Historical Immutability & Auditability 14

- SaleItem, SaleReturn, SaleReturnItem, Exchange, InventoryMovement محفوظين كتاريخ immutable.
- Sale تسمح فقط بالتحول VOIDED → COMPLETED مع metadata محددة؛ باقي fields التاريخية/المالية لا تتغير.
- Direct cross-tenant lifecycle links مرفوضة حتى عبر SQL.

Prisma & Migration Workflow .15

```
schema.prisma
-> reviewed migration.sql
-> inspect SQL
-> prisma migrate deploy
-> Supabase PostgreSQL
-> prisma generate
-> tests / typecheck / build
-> commit
```

- لم نعد نحتاج Copy/Paste للمigration داخل Supabase SQL Editor.
- كل migration تُراجع قبل deploy، و Prisma يسجل المطبّق داخل prisma_migrations.

16. DailyReport - آخر نقطة قيد العمل

DailyReport هو rebuildable cache وليس مصدر الحقيقة. المصدر الحقيقي هو

.void metadata و Sale/SaleItem/SaleReturn/SaleReturnItem

النسخة الأولى من redesign كانت ناقصة لأنها حسبت netProfit بدون operatingExpenses. القرار الحالي قبل

الdeploy هو الحفاظ على الفصل الصحيح بين Gross Profit و Net Profit:

```
grossRevenue
returnedRevenue
voidedRevenue
netRevenue = grossRevenue - returnedRevenue - voidedRevenue

grossCOGS
returnedCOGS
voidedCOGS
netCOGS = grossCOGS - returnedCOGS - voidedCOGS

grossProfit = netRevenue - netCOGS
operatingExpenses = daily Expense aggregate
netProfit = grossProfit - operatingExpenses
```

- Reversals تُنسب لتاريخ حدوثها، لذلك netRevenue / netCOGS / profit قد تكون سالبة في يوم معيّن.
- الـ pending migration لازم يتعدل بهال semantics قبل الـ deploy.

17. ما تبقى بعد DailyReport

الحالة	المكوّن
التالي مباشرة ⌚	DailyReport accounting correction + deploy
⌚	Sharp + Cloudflare R2 backend integration
⌚	Products + Variants backend API
⌚	Products UI + image upload
⌚	Inventory / Restock runtime
⌚	Sales / POS runtime
⌚	Return / Void / Exchange APIs + UI
⌚	Reporting queries + rebuild logic + dashboard
⌚	WAREHOUSE invitations/lifecycle
⌚	Deployment: Netlify + Railway

18. الخلاصة

المشروع تجاوز مرحلة الـ authentication والـ tenant setup وصار عنده database lifecycle قوي يحمي التاريخ المالي والمخزني. Categories مكتملة بالكامل، Product schema الأساسية شبه ثابتة، و Cloudflare R2 مُجهز. آخر خطوة قبل العودة للـ Products هي تثبيت DailyReport accounting semantics ثم نشر migration الخاصة بها.